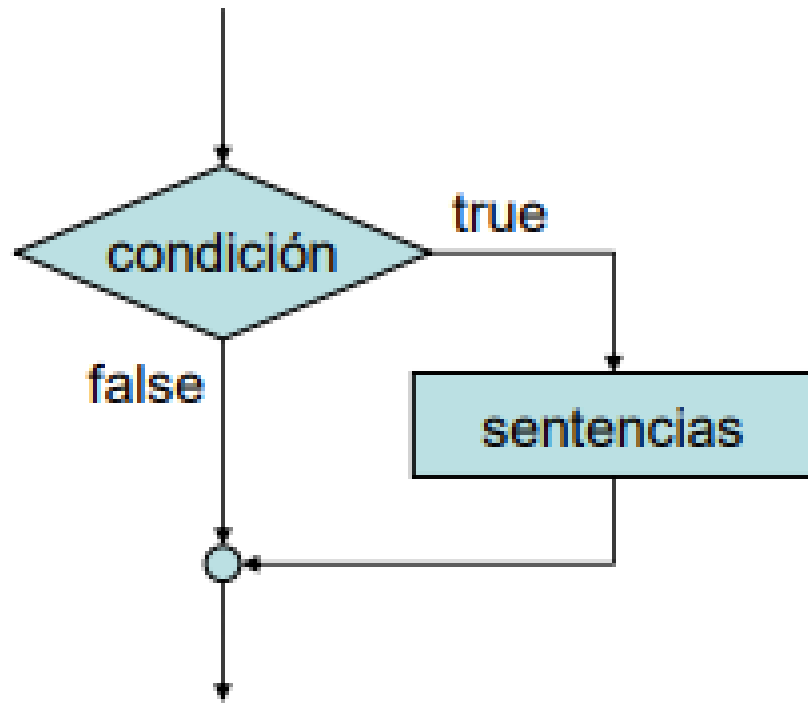

ESTRUCTURAS DE CONTROL

UD2

ESTRUCTURA DE SELECCIÓN - IF

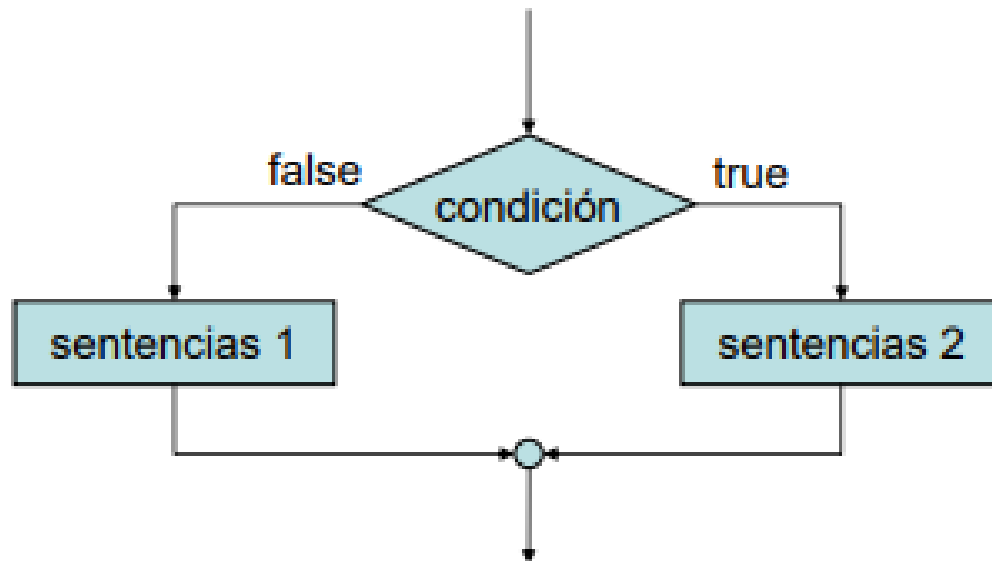


```
if (condición) {  
    create a scope  
    // instrucciones si la condición es verdadera  
}
```

Ejemplo:

```
public class IfExample {  
    public static void main(String[] args) {  
        int edad = 18;  
        if (edad >= 18) {  
            System.out.println("Eres mayor de edad.");  
        }  
    }  
}
```

ESTRUCTURA DE SELECCIÓN – IF-ELSE



```
if (condición) {  
    // instrucciones si la condición es verdadera  
} else {  
    // instrucciones si la condición es falsa  
}
```

Ejemplo:

```
public class IfElseExample {  
    public static void main(String[] args) {  
        int edad = 16;  
        if (edad >= 18) {  
            System.out.println("Eres mayor de edad.");  
        } else {  
            System.out.println("No eres mayor de edad.");  
        }  
    }  
}
```

ENUM

Uso de Enum en Java

- Un **enum** (enumeración) en Java es un tipo especial de clase que limita los valores posibles para una variable a un conjunto fijo y predefinido de constantes.
- Estas constantes son objetos estáticos y final, definidos en mayúsculas por convención.
- Los enums mejoran la legibilidad y la seguridad del código al evitar el uso de valores arbitrarios o erróneos.

```
public enum NombreEnum {  
    CONSTANTE1,  
    CONSTANTE2,  
    CONSTANTE3;  
}
```

Ejemplo:

```
public enum Day {  
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY,  
    THURSDAY, FRIDAY, SATURDAY  
}
```

ESTRUCTURA DE SELECCIÓN – SWITCH

VERSIÓN ANTERIOR

```

switch (expresión) {
    case valor1:
        // instrucciones para valor1
        break; Break the scope.
        Finish and go out of the switch
    case valor2:
        // instrucciones para valor2
        break;
    // ...
    default:
        // instrucciones si no coincide ningún caso
}

```

Ejemplo:

```

public class SwitchEnumExample {

    public static void main(String[] args) {

        Day today = Day.WEDNESDAY; name of class.value

        switch (today) {

            case SUNDAY:

                System.out.println("Sunday is a day of rest."); break;

            case MONDAY:

                System.out.println("Monday blues."); break;

            case TUESDAY:

                System.out.println("Tuesday is productive."); break;

            case WEDNESDAY:

                System.out.println("Wednesday is midweek."); break;

            case THURSDAY:

                System.out.println("Thursday is almost Friday."); break;

            case FRIDAY:

                System.out.println("Friday is party time!"); break;

            case SATURDAY:

                System.out.println("Saturday is for fun."); break;

            default:

                System.out.println("Unknown day."); break;

        }

    }

}

```

ESTRUCTURA DE SELECCIÓN – SWITCH

VERSIÓN NUEVA

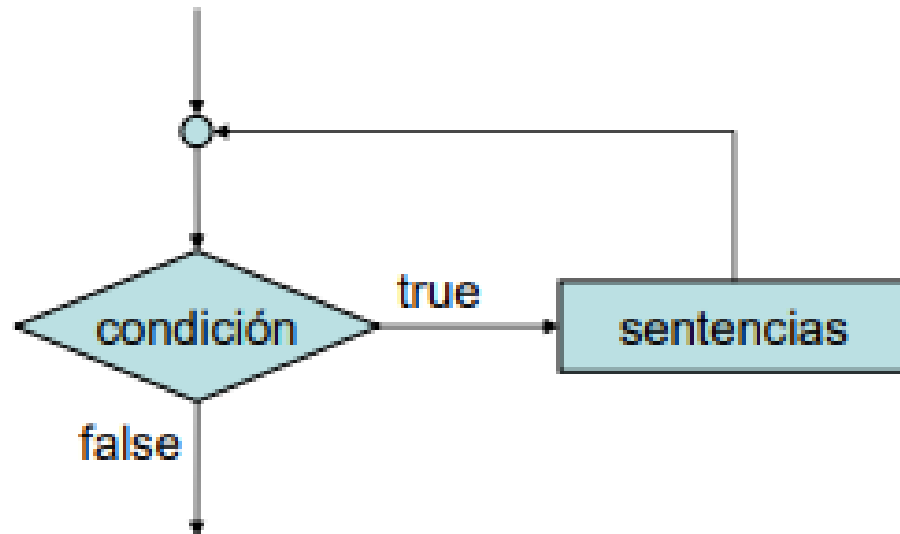
```
switch (expresión) {  
    case valor1 -> instrucción1;  
    case valor2 -> instrucción2;  
    default -> instrucciónDefault;  
}
```

Ejemplo:

```
public class YieldSwitchExample {  
    public static void main(String[] args)  
{ int day = 3; String dayName = switch (day)  
    { case 1 -> "Monday";  
      case 2 -> "Tuesday";  
      case 3 -> { System.out.println("Processing day 3");  
                 yield "Wednesday"; // yield devuelve el valor para  
                 asignar a dayName }  
      case 4 -> "Thursday";  
      default -> "Unknown day"; }; System.out.println("Day is: " +  
dayName); }  
}  
}
```

ESTRUCTURA DE REPETICIÓN – WHILE

If the condition is false from the start, the body may never execute even once



```
while (condición) {
```

```
    // instrucciones a repetir mientras condición sea verdadera
```

```
}
```

Ejemplo:

```
public class WhileExample {
```

```
    public static void main(String[] args) {
```

```
        int i = 1;
```

```
        while (i <= 3) {
```

```
            System.out.println(i);
```

```
            i++;    need to increment,  
                   if not an infinite loop
```

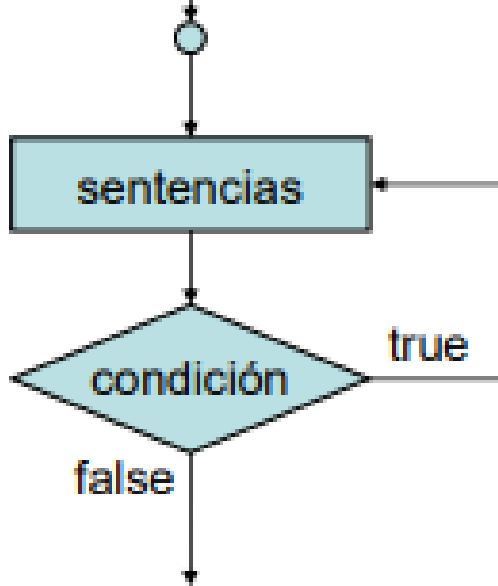
```
        }
```

```
    }
```

```
}
```

ESTRUCTURA DE REPETICIÓN – DO_WHILE

loop body is executed at least once, regardless of the condition.
When you want the code block to always run at least once



```
do {  
    // instrucciones a repetir  
} while (condición);
```

Ejemplo:

```
public class DoWhileExample {  
    public static void main(String[] args) {  
        int i = 1;  
        do {  
            System.out.println(i);  
            i++;  
        } while (i <= 3);  
    }  
}
```


ESTRUCTURA DE SELECCIÓN – FOR

```
for (inicialización; condición; actualización) {  
    // instrucciones a repetir mientras condición sea verdadera  
}
```

Ejemplo:

```
public class ForExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            System.out.println(i);  
        }  
    }  
}
```

ESTRUCTURA DE SELECCIÓN – BREAK

break;

Descripción:

Permite salir inmediatamente de un bucle o switch.

Diagrama de flujo:

Durante la iteración o evaluación, si se encuentra break, sale del ciclo/switch y continúa después.

Ejemplo:

```
public class BreakExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 5; i++) {  
            if (i == 3) break;  
            System.out.println(i);  
        }    only prints 1, 2  
    }  
}
```

ESTRUCTURA DE SELECCIÓN – CONTINUE

continue;

Descripción:

Salta la iteración actual y continúa con la siguiente del bucle.

Diagrama de flujo:

Durante la iteración, si se encuentra continue, se omiten las instrucciones restantes en la iteración actual y se pasa a la siguiente.

Ejemplo:

```
public class ContinueExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 5; i++) {  
            if (i == 3) continue;  
            System.out.println(i);  
        }  
    }  
}
```

건너뛰기, 아래로 내려가지 않고 위로 올라감
print 1, 2, 4, 5